

Differentiable Simulation on DiffSim:
Three Ways to a Gradient, the Discrete Adjoint,
and the Recipe
The E0 course document (Track E, adjoint on-ramp)

The DiffSim curriculum

July 2026

Abstract

This document is the theory companion to the three runnable E0 tutorials (`tutorials/E_differentiable/E0a--E0c`). It develops the mathematics of differentiable simulation in DiffSim notation, consolidates the theory of the three chapters, and closes with the Adjoint-Readiness Checklist as an appendix. Structure and example lineage are adapted from the mathematical background of **dolfin-adjoint/pyadjoint** by Patrick E. Farrell; the mother problem, three-way comparison device, and “learning reaction rates” examples follow Farrell, Ham, Funke & Rognes [1] and Mitusch, Funke & Dokken [2]. DiffSim-native content — the discrete-adjoint stance, taping rules, the beyond-FH gauge story, the H4 signal-scale protocol — is developed in the E-track specs and dev notes. House rule: *theory predicts a number, then we print the measured number next to it*. All numbers quoted here are from the gate batteries of tutorials E0a–E0c, reproducible on CPU in under three minutes each.

Contents

Part I: The Reduced Functional	3
1 The reduced functional	3
2 Three ways to $\nabla_m \hat{J}$	4

2.1	Finite differences (FD)	4
2.2	Tangent-linear method (TLM, forward-mode AD)	4
2.3	Adjoint (reverse-mode)	4
3	Information-flow reversal	5
	Part II: Discretize-Then-Differentiate	6
4	The discrete-adjoint stance	6
5	The Lagrangian derivation of the discrete adjoint	7
	Part III: Anatomy of a Taped Brick	8
6	What <code>wp.Tape</code> records	8
7	The do-not-differentiate list	8
8	Compute-once-and-store patterns	9
9	The dot-product test	10
	Part IV: Transient Chains	10
10	The transient adjoint	10
11	Compute-vs-memory: full-store and recompute	11
	Part V: Choosing a Good J	12
12	Smoothness: nonsmooth J and its relaxation	12
13	Signal scale: the H4 protocol	12
14	Flat- J geometry and informative probe placement	13
15	Identifiability, gauge freedom, and regularisation as science	13

16 Multi-observable J and the SP-1 bridge	14
Part VI: Mini Phase-Field Payoff	15
17 Allen-Cahn: learning mobility and gradient-energy coefficient	15
Part VII: Numerical Results Summary	16
18 Measured gates	16
A The Adjoint-Readiness Checklist	17

Part I: The Reduced Functional and Three Paths to Its Gradient

1 The reduced functional

Let $m \in \mathbb{R}^N$ be a vector of simulation parameters (a conductivity field per element, material coefficients, boundary data, ...). The forward model maps m to a state $u(m) \in \mathbb{R}^n$ by solving a residual equation

$$R(u(m), m) = 0. \quad (1)$$

A scalar quantity of interest $J(u, m)$ defines the *reduced functional*

$$\hat{J}(m) = J(u(m), m), \quad (2)$$

where $u(m)$ is understood as the implicit solution of (1). The optimisation problem is $\min_m \hat{J}(m)$; gradient-based methods require $\nabla_m \hat{J}$.

The mother problem (Farrell/pyadjoint). Throughout Part I we use the Poisson conductivity problem as a concrete instance: $-\nabla \cdot (\kappa(x) \nabla u) = f$ on $\Omega = [0, 1]^2$ with $u|_{\partial\Omega} = g$. The conductivity field $\kappa(x)$ is our parameter, discretized as a per-element constant $m = (\kappa_1, \dots, \kappa_N)$ on a uniform Q_1 mesh. Someone hands us probe measurements $u_{\text{obs}}(x_i)$ at P interior locations; the objective is

$$J = \frac{1}{2} \sum_{i=1}^P (u(x_i) - u_{\text{obs},i})^2. \quad (3)$$

For $N = 64$ elements (level-3 grid) the tutorial measures $\hat{J}(\kappa_{\text{eval}}) \approx 8.35 \times 10^{-3}$ at a uniform evaluation point $\kappa_{\text{eval}} = 1.3$, with a two-blob true field $\kappa_{\text{true}} \in [1.0, 2.5]$.

2 Three ways to $\nabla_m \hat{J}$

Differentiating the residual constraint (1) implicitly links $\nabla_m \hat{J}$ to u and R . Three methods exploit this link at radically different cost.

2.1 Finite differences (FD)

For each parameter index $i = 1, \dots, N$:

$$\frac{\partial \hat{J}}{\partial m_i} \approx \frac{\hat{J}(m + \epsilon e_i) - \hat{J}(m - \epsilon e_i)}{2\epsilon}. \quad (4)$$

Cost: $2N$ **forward solves** (central differences). Accuracy: limited by floating-point cancellation for $\epsilon \lesssim \sqrt{\epsilon_{\text{machine}}}$. At $N = 64$ this requires 128 forward solves.

2.2 Tangent-linear method (TLM, forward-mode AD)

Differentiate (1) with respect to m_i :

$$\frac{\partial R}{\partial u} \dot{u}_i + \frac{\partial R}{\partial m_i} = 0 \quad \Rightarrow \quad \dot{u}_i = - \left(\frac{\partial R}{\partial u} \right)^{-1} \frac{\partial R}{\partial m_i}. \quad (5)$$

Each $\dot{u}_i$ is obtained by one *forward-mode* linear solve, then $\partial \hat{J} / \partial m_i = (\partial J / \partial u) \dot{u}_i$. Cost: N **forward-mode solves** — half the FD cost, same $O(N)$ scaling.

2.3 Adjoint (reverse-mode)

Form the Lagrangian by appending the residual constraint with a multiplier λ :

$$\mathcal{L}(u, m, \lambda) = J(u, m) - \lambda^\top R(u, m). \quad (6)$$

Stationarity with respect to u gives the **adjoint equation**:

$$\frac{\partial \mathcal{L}}{\partial u} = 0 \quad \Rightarrow \quad \left(\frac{\partial R}{\partial u} \right)^\top \lambda = \left(\frac{\partial J}{\partial u} \right)^\top. \quad (7)$$

This is *one linear system for λ , independent of N* . Once λ is known, the gradient with respect to every parameter follows from $\partial\mathcal{L}/\partial m = 0$:

$$\frac{\partial\hat{J}}{\partial m} = \frac{\partial J}{\partial m} - \lambda^\top \frac{\partial R}{\partial m}. \quad (8)$$

Cost: **1 forward solve + 1 adjoint solve**, regardless of N .

Remark 1 (adjoint linearity). Even when $R(u, m) = 0$ is nonlinear in u , the adjoint equation (7) is *always linear* in λ — it is the transposed Jacobian of R evaluated at the converged state $u^*(m)$. There is no iteration, only one linear solve.

Table 1: Three paths to $\nabla_m \hat{J}$, cost in forward-solve units. At $N = 64$ the tutorial measures an FD/adjoint wall-time ratio of approximately 30–80 $\times$ (kernel compilation dominates the first run; steady-state ratio $\approx 35\times$).

Method	Solve cost	Exact?	Scales with N ?
Finite differences (central)	$2N$ forward	No (ϵ -limited)	Yes
Tangent-linear	N forward	Yes	Yes
Adjoint	$1 + 1$	Yes	No

For the Poisson mother problem with $N = 64$ the adjoint equation is $A^\top \lambda = (\partial J / \partial u)^\top$, where $A(\kappa)$ is the assembled constrained stiffness. Since A is symmetric ($A^\top = A$), the forward-solve LU factorization can be reused directly for the adjoint solve at no additional cost.

Measured. Level-3 grid, 64 elements, 5 probes, FP64. Adjoint vs central FD: max relative error $< 10^{-6}$. Adjoint vs `wp.Tape` (taped in κ): max relative error $< 10^{-9}$. FD wall time / adjoint wall time: 30–80 $\times$ (first run); $\approx 35\times$ steady-state. (`tutorials/E_differentiable/E0a_thinking_differentiable.py`)

3 Information-flow reversal

The forward solve propagates information *from* the parameter field m *through* the PDE *to* the output functional J . The adjoint reverses this flow: it carries sensitivity *from* the scalar J *backward through* the PDE *to* every parameter

at once. This is why the adjoint cost is $O(1)$ in N : instead of differentiating N forward paths, it differentiates one backward path.

The information-flow picture is due to Farrell [1] and is the central image of the E0a tutorial.

Part II: Discretize-Then-Differentiate

4 The discrete-adjoint stance

Two philosophies exist for adjoints of PDE-constrained problems.

Differentiate-then-discretize derives the adjoint PDE in the continuous setting (a second boundary-value problem) and then discretizes it. The resulting discrete adjoint is not the exact transpose of the primal discretization; it introduces an additional error term of the same order as the spatial discretization.

Discretize-then-differentiate first commits to a discrete residual $R_h(u_h, m) = 0$ and then differentiates the discrete system. The resulting discrete adjoint satisfies $(\partial R_h / \partial u_h)^\top \lambda = (\partial J_h / \partial u_h)^\top$ *exactly*—there is no extra approximation beyond the chosen discretization.

DiffSim’s stance is discretize-then-differentiate (spec S4.3). Every formula in these notes applies to the discrete system; the word “adjoint equation” always refers to the discrete transpose.

Remark 2 (IFT / relation vs. algorithm). The Implicit Function Theorem (IFT) underlies the entire approach. Given a smooth residual $R(u, m) = 0$ with $\partial R / \partial u$ invertible at a solution (u^*, m) , the IFT guarantees a locally smooth map $m \mapsto u^*(m)$ whose derivative is

$$\frac{\partial u^*}{\partial m} = - \left(\frac{\partial R}{\partial u} \right)^{-1} \frac{\partial R}{\partial m} \Big|_{(u^*, m)}. \quad (9)$$

We differentiate the *relation* $R = 0$, not the *algorithm* used to solve it. A Newton iteration, a Krylov method, or an LU factorization is an algorithm for inverting R ; none of these need to be differentiated. Only the assembled matrix $\partial R / \partial u$ evaluated at convergence enters the adjoint equation.

5 The Lagrangian derivation of the discrete adjoint

Let $R_h : \mathbb{R}^n \times \mathbb{R}^N \rightarrow \mathbb{R}^n$ be the discrete residual and $J_h : \mathbb{R}^n \times \mathbb{R}^N \rightarrow \mathbb{R}$ the discrete objective. The Lagrangian is

$$\mathcal{L}(u_h, m, \lambda) = J_h(u_h, m) - \lambda^\top R_h(u_h, m). \quad (10)$$

Stationarity conditions:

$$\frac{\partial \mathcal{L}}{\partial \lambda} = 0 \implies R_h(u_h^*, m) = 0 \quad (\text{forward problem}), \quad (11)$$

$$\frac{\partial \mathcal{L}}{\partial u_h} = 0 \implies \underbrace{\left(\frac{\partial R_h}{\partial u_h} \right)^\top}_{A^\top} \lambda = \underbrace{\left(\frac{\partial J_h}{\partial u_h} \right)^\top}_{\text{RHS}} \quad (\text{adjoint equation}), \quad (12)$$

$$\frac{\partial \mathcal{L}}{\partial m} = 0 \implies \frac{\partial \hat{J}}{\partial m} = \frac{\partial J_h}{\partial m} - \lambda^\top \frac{\partial R_h}{\partial m} \quad (\text{gradient formula}). \quad (13)$$

For the Poisson problem, $\partial R_h / \partial u_h = A(\kappa)$ and $\partial R_h / \partial \kappa_e = A_e$ (the element stiffness at $\kappa = 1$), so (13) reduces to

$$\frac{\partial \hat{J}}{\partial \kappa_e} = -\lambda_{\text{int}}^\top A_e u_h^*, \quad (14)$$

where λ_{int} is λ with boundary-node entries zeroed (because the Dirichlet-eliminated rows of A do not depend on κ_e).

Remark 3 (symmetry and factorization reuse). For the Poisson operator $A = A^\top$, so the adjoint equation $A^\top \lambda = \partial J_h / \partial u_h$ is solved with the *same LU factorization* as the forward solve. In `scipy.sparse.linalg.splu`, the call `lu.solve(rhs, trans='T')` triggers the transposed forward-backward substitution using the stored triangular factors. For non-symmetric operators (SBM, convection-diffusion, Navier-Stokes) the same principle holds: one factorization, two solve directions.

Part III: Anatomy of a Taped Brick

6 What `wp.Tape` records

A Warp `wp.Tape` is a *launch graph*: a list of

(`kernel_ref`, `dim`, `input arrays`, `output arrays`)

tuples appended during the `with tape:` scope. On `tape.backward()`, the list is replayed in reverse; for each entry the backward kernel propagates cotangents

$$\text{adj_}x += \frac{\partial f}{\partial x} \text{adj_}y,$$

where `adj_y = tape.gradients[y]` is the incoming seed. Arrays with `requires_grad=False` are *frozen*: the tape skips cotangent accumulation for them.

Custom VJP vs. unrolled tape. When a kernel has `enable_backward=False` (e.g. the Poisson assembly kernel in `diffsim/assembly/operators.py`), the tape does *not* generate an automatic backward kernel. The gradient path is instead supplied as a *custom VJP* — in DiffSim’s case, the transposed solve $A^\top \lambda = \partial J / \partial u$. This is intentional: letting the tape unroll an LU factorization would be catastrophically expensive and unnecessary, given the IFT.

7 The do-not-differentiate list

Each item below is an *algorithm* implementing a mathematical *relation*. The relation has a clean adjoint; the algorithm’s code graph does not.

1. **Linear solvers (`splu`, `bicgstab`, `GMRES`, `cuDSS`, ...).** Relation: $A(m)u = b$. Adjoint: $A^\top \lambda = \partial J / \partial u$ (one solve, same LU factors). Never unroll the LU/Krylov steps through a tape.
2. **Newton / nonlinear iterations.** Relation: $R(u^*, m) = 0$ defines $u^*(m)$ implicitly (IFT). Adjoint: $(\partial R / \partial u)^\top \lambda = \partial J / \partial u$ evaluated at the *converged* state. Never tape through the iteration loop.
3. **Mesh build, octree carve, element classification, preflight checks.** These produce combinatorial objects (connectivity tables, sorted element lists) that are piecewise constant in the geometric parameters;

their derivative is zero almost everywhere and undefined at transitions. The correct approach: *freeze* the mesh before opening the tape. Geometric sensitivities live in the boundary parametrization (shape derivatives), not the mesh arrays. E1 (`E1_shape_optimization.py`) is the worked instance.

4. **RNG seeds and tabulated basis functions.** Basis tables (N, dN, w) are computed once by `basis_tables(p, dim)` and treated as frozen inputs with `requires_grad=False`. Random seeds are inputs in the reparametrization sense; they are never inside the tape scope.

8 Compute-once-and-store patterns

Three canonical patterns avoid redundant computation.

(i) **One LU factorization for A and A^T solves.** `scipy.sparse.linalg.splu` stores both triangular factors; the forward solve (`lu.solve(b)`) and the adjoint solve (`lu.solve(c, trans='T')`) share the same factorization.

Measured. Level-3 Poisson, 50 trials. One LU (two solve calls): $\approx t$ ms. Two separate LU factorizations: $\approx 1.7t$ ms. Ratio $> 1.2\times$ (measured; the factorization cost dominates at this size; the ratio grows toward $2\times$ at scale because factorization is $O(n^{1.5})$ in 2-D while each solve is $O(n \log n)$). (E0b, §4.)

(ii) **Tabulated basis reuse.** `dm.bins[p]['dN']` is computed once; every kernel launch takes a reference to the same device array. Its shape is (nqp, nb, dim) — at Q_1 on a 2-D level-3 grid this is $(4, 4, 2)$.

(iii) **GP-field precomputation (*frozen at GPs*).** A closure field $\kappa_\theta(x)$ is evaluated at all Gauss points *before* the tape scope, stored in an array with `requires_grad=False`, and passed as a frozen input to the assembly kernel. The tape tracks only the PDE residual’s dependence on the tabulated values, not the internals of κ_θ . This is the M2 “one-way coupler” convention: the closure changes in the outer optimisation loop; the PDE adjoint needs only $\partial R / \partial \kappa$ at the quadrature points.

9 The dot-product test

For any operator A and vectors v, w :

$$\langle Av, w \rangle = \langle v, A^\top w \rangle. \quad (15)$$

This is the cheapest possible sanity check for a new operator and its adjoint. The test does not require an explicit matrix: only the forward action $v \mapsto Av$ and the adjoint action $w \mapsto A^\top w$ are needed (it is valid for matrix-free operators). A failure means the adjoint is wrong; a pass to machine precision gives strong evidence it is correct.

The verification protocol for every new DiffSim operator: pick random v, w ; compute Av and $A^\top w$; check

$$\frac{|\langle Av, w \rangle - \langle v, A^\top w \rangle|}{\|Av\| \|w\| + 10^{-100}} < 10^{-12}.$$

Measured. Level-3 Poisson assembled CSR. Relative error of dot-product test: $\sim 10^{-17}$ (Poisson is symmetric, $A = A^\top$, so the test is algebraically exact up to floating-point rounding). (E0b, §5.)

Part IV: Transient Chains and Compute-vs-Memory

10 The transient adjoint

A time-marching scheme produces a sequence of states $u^0, u^1, \dots, u^N$. Each step is a residual

$$F^n(u^n, u^{n-1}, m) = 0, \quad n = 1, \dots, N. \quad (16)$$

For BDF1 heat: $F^n = A u^n - (M/\Delta t) u^{n-1} = 0$, with $A = M/\Delta t + K(\kappa)$. The Lagrangian appends one multiplier λ^n per step:

$$\mathcal{L} = J_h(u^N, m) - \sum_{n=1}^N (\lambda^n)^\top F^n(u^n, u^{n-1}, m). \quad (17)$$

Stationarity at u^n for $n < N$:

$$\begin{aligned} \left(\frac{\partial F^n}{\partial u^n} \right)^\top \lambda^n &= - \left(\frac{\partial F^{n+1}}{\partial u^n} \right)^\top \lambda^{n+1} \\ A^\top \lambda^n &= P^\top \lambda^{n+1}, \quad P = M/\Delta t, \end{aligned} \quad (18)$$

with terminal condition at $n = N$:

$$A^\top \lambda^N = \left(\frac{\partial J_h}{\partial u^N} \right)^\top = W^\top (W u^N - u_{\text{obs}}). \quad (19)$$

This is the *adjoint recurrence*: solve backward in time, one transposed solve per step. The gradient contribution from step n is

$$\Delta \left(\frac{\partial \hat{J}}{\partial \kappa} \right)_e += -(\lambda_{\text{int}}^n)^\top A_e u^n, \quad (20)$$

where u^n is the *primal state stored from the forward march* (checklist item 3 in Appendix A).

Remark 4 (one factorization, all steps). When κ is constant in time, $A = M/\Delta t + K(\kappa)$ does not change between steps. A *single LU factorization* serves every forward solve and every adjoint solve — forward via `lu.solve(rhs)` and backward via `lu.solve(seed, trans='T')`. Both directions reuse the same stored triangular factors.

Measured. BDF1 heat, level-3 grid, 30 steps, $\Delta t = 0.01$. Transient-chain check (discrete adjoint vs central FD, end-to-end): max relative error $< 10^{-5}$. (E0c, §2.)

11 Compute-vs-memory: full-store and recompute

The backward sweep (20) requires u^n at each step. Two strategies supply it.

Full-store. Keep $u^0, u^1, \dots, u^N$ in memory. Memory: $(N + 1) \times n_{\text{nodes}} \times 8$ bytes (FP64). Backward cost: N transposed solves. At 10^6 dofs and 10^3 steps this is 8 GB — a regime where full-store exceeds GPU memory.

Recompute from checkpoints. Store n_c checkpoints at intervals of $\Delta n = N/n_c$ steps. When the backward sweep needs u^n , re-march forward from the nearest earlier checkpoint. Memory: $n_c \times n_{\text{nodes}} \times 8$ bytes. Backward cost: N transposed solves plus $O(n_c \cdot \Delta n)$ extra forward solves.

The optimal checkpoint schedule is *revolve* (Griewank & Walther [3]): $O(\log N)$ checkpoints achieve $O(N \log N)$ recompute at $O(\log N)$ memory, provably

optimal. Binomial scheduling libraries (`pyrevolve`, `checkpoint_schedules`) implement this and wire directly into the backward loop above.

The $2.5\times$ cost contract. A correct reverse-mode adjoint adds at most one transposed solve plus one gradient contraction per forward step. The backward march should cost $\leq 2.5\times$ the forward march. If backward $\gg$ forward, something is being unrolled that should not be.

Measured. BDF1 heat, level-3, 30 steps. Full-store backward / forward wall-time ratio $\leq 2.5\times$ (measured). Primal-storage inventory: 31 states $\times$ 81 nodes $\times$ 8 bytes = 20 KiB. Full-store vs recompute- from-6-checkpoints gradient agreement: $< 10^{-10}$ (same discrete adjoint, bit-for-bit identical u^n from re-march). (E0c, §3.)

Part V: Choosing a Good J

12 Smoothness: nonsmooth J and its relaxation

Functionals built on $\max$, $\min$, $|\cdot|$, or hard thresholds are nonsmooth: the gradient jumps when the active branch switches, and is zero almost everywhere for a threshold. An optimiser fed a jumpy gradient stalls or oscillates.

The standard cure is a *relaxed surrogate*. For $J = \max_i g_i(p)$, the logsumexp (softmax) upper bound

$$J_\beta(p) = \frac{1}{\beta} \log \sum_i \exp(\beta g_i(p)) \xrightarrow{\beta \rightarrow \infty} \max_i g_i(p) \quad (21)$$

has a smooth softmax-weighted gradient at any finite β .

Measured. Sweeping a scalar p over $[0, 3]$: max-J discrete gradient maximum jump ≈ 0.55 ; logsumexp ($\beta = 40$) maximum jump ≈ 0.17 ; ratio $\approx 0.31 < 0.5$ (smoother by $1.8\times$). (E0c, §4(i).)

13 Signal scale: the H4 protocol

Before launching any optimiser, verify that $|\partial \hat{J} / \partial m|$ clears the finite-difference noise floor. The *H4 protocol* (DiffSim-native): compute central FD across a

range of ϵ values and take the minimum FD spread as the noise-floor estimate. If the adjoint signal does not exceed the floor by orders of magnitude, the objective barely responds to the parameters — rescale m or J , or choose a more informative observable, before spending a single optimiser iteration.

Measured. Heat chain, level-3. Adjoint signal $|\partial\hat{J}/\partial\kappa|_\infty \approx 10^{-3}$; FD noise floor (min spread across $\epsilon \in [10^{-3}, 10^{-8}] \approx 10^{-7}$; signal-to-floor ratio $\approx 10^4 \gg 10^3$ (required). (E0c, §4(ii).)

14 Flat- J geometry and informative probe placement

A probe placed where the solution field has negligible amplitude sees $\partial u/\partial m \approx 0$: the gradient is near zero regardless of how good the adjoint is. The observable placement is part of the experiment design, not the solver.

Measured. Heat chain, sharp bump at $(0.25, 0.25)$. Probe on the active bump: $\|\partial\hat{J}/\partial\kappa\|_\infty \approx 2 \times 10^{-3}$. Probe in the far corner $(0.84, 0.84)$: $\approx 2 \times 10^{-5}$. Contrast $\approx 100\times \gg 10\times$ (required). (E0c, §4(iii).)

15 Identifiability, gauge freedom, and regularisation as science

A gradient can be correct, large, and smooth while the inverse problem remains ill-posed: the parameterisation may have *unidentifiable directions* — parameter combinations that produce zero observable change. The diagnostic is the spectrum of the observation Jacobian (the Gramian $G = J_m^\top J_m$ where $J_m = \partial\mathbf{d}/\partial m$ maps parameters to data), not the gradient of J itself.

Case study: the beyond-FH gauge story (M4, rung 2). Learning a free-energy correction $f'_\theta(c)$ on top of Flory-Huggins has two structurally invisible modes. A constant in f' shifts the chemical potential μ by a constant, which is invisible to conserved- c dynamics. The linear mode proportional to $(1 - 2c)$ is *exactly aliased* with the FH chi term: the trajectory difference along this direction was measured at 5×10^{-15} . These are *gauge freedoms*; they must be removed from the parameterisation before fitting.

Quantitatively (numbers quoted from the beyond-FH dev note [4]):

- Un-anchored basis $\{\chi, P_1, P_2, P_3\}$: observation-Jacobian condition number $\approx 1.4 \times 10^{16}$ (χ and P_1 aliased — effectively singular).
- Anchored basis $\{\chi, P_2, P_3\}$ (Legendre modes orthogonal to $\{1, c\}$, removing the gauge by construction): condition number ≈ 18 (well-posed). Recovery of coefficients from a single interior trajectory: relative error $\approx 5.6 \times 10^{-8}$.
- Conditioning improvement: $\approx 7.6 \times 10^{14}$.

Even with a gauge-anchored parameterisation, a *single narrow trajectory* (visited composition range 0.46–0.56) leaves higher modes weakly constrained (condition number $\approx 2 \times 10^2$ – 5×10^2). A *composition-diverse* protocol (range 0.21–0.85) reduces the condition number to ≈ 14 – 18 (15 – $30\times$ improvement) and recovers $\{P_2, \dots, P_5\}$ to coefficient error $\approx 4 \times 10^{-5}$, while the single narrow fit fails (coefficient error ≈ 0.9 – 1.0 , a $> 2 \times 10^4\times$ recovery gap).

Tikhonov regularisation as a prior, not a hack. A penalty $\varepsilon\|\theta\|^2$ encodes the hypothesis that the correction is small. In the beyond-FH fit it does not merely stabilise: it pulls the ill-conditioned single-trajectory fit to a definite (wrong) answer rather than to NaN, making the failure legible. Regularisation states a scientific hypothesis; the condition number of the observation Jacobian tells you whether the data can test it.

Remark 5 (the rule taught three times). E2 (the recovery plateau), M2–M3 (the observability ladder), and the beyond-FH study all arrive at the same principle: *mechanism before optimiser*. When a fit stalls, measure identifiability before tuning the descent.

16 Multi-observable J and the SP-1 bridge

A single scalar probe misfit is often flat or aliased (§§14 and 15). The cure is a richer, *instrument-space* observable — or several:

$$J = \sum_k w_k \|\mathcal{O}_k(u) - d_k\|^2, \quad (22)$$

where $\mathcal{O}_k$ are diverse operators (probe values, a structure factor $S(q, t)$, a film height $h(t)$, a PSF-convolved image). Diverse protocols illuminate different directions of parameter space, turning the underdetermined inverse

of E0b (5 probes for 64 unknowns) into a well-posed one. The beyond-FH instrument-space $S(q)$ misfit backpropagates to the closure coefficients matching FD to 1.8×10^{-11} (structure factor computed as a differentiable Warp kernel). SP-1 (R3) builds the multi-observable, multi-protocol inversion on the production stack.

Part VI: The Mini Phase-Field Payoff

17 Allen-Cahn: learning mobility and gradient-energy coefficient

The Allen-Cahn equation

$$\partial_t c = -M[f'(c) - \kappa \Delta c], \quad f(c) = \frac{1}{4}(c^2 - 1)^2, \quad (23)$$

is the group’s own physics (F-track, M4) and serves as the payoff example for the E0c recipe. The “learning reaction rates” framing is the dolfin-adjoint tutorial of that name [2]; we adapt it to the Warp discrete-adjoint stack.

We differentiate $J = \frac{1}{2}\|Wc^N - c_{\text{obs}}\|^2$ with respect to the scalar mobility M and the gradient-energy coefficient κ through a 25-step semi-implicit BDF1 march. The adjoint recurrence (18) takes the form

$$A^\top \lambda^n = \text{seed}^n, \quad (24)$$

$$\text{seed}^{n-1} = -\frac{\partial F^n}{\partial c^{n-1}} \lambda^n = \left[\frac{m_\ell}{\Delta t} - M m_\ell (3(c^{n-1})^2 - 1) \right] \lambda^n, \quad (25)$$

where m_ℓ is the lumped mass diagonal.

The three-way gradient check uses central FD as one leg and *complex-step differentiation* as the second exact leg: for a holomorphic extension $\hat{J}(m + i h)$ with $h = 10^{-30}$,

$$\frac{\partial \hat{J}}{\partial m} = \frac{\text{Im } \hat{J}(m + i h)}{h} + O(h^2), \quad (26)$$

with no subtraction error. Agreement of all three legs to $< 10^{-6}$ relative error provides strong evidence of correctness independent of any single computational path.

Measured. Allen-Cahn, level-3, 25 steps, $\Delta t = 0.005$. Three-way check: adjoint vs complex-step vs FD, max relative error (worst of $\partial \hat{J}/\partial M$ and $\partial \hat{J}/\partial \kappa$) $< 10^{-6}$. (E0c, §5.)

Part VII: Numerical Results Summary

18 Measured gates

Table 2: Measured gate results from tutorials E0a–E0c (CPU, FP64, level-3 grid unless otherwise noted). All reproducible with `python tutorials/E_differentiable/E0{a,b,c}*.py`.

Chapter	Gate	Required	Measured
E0a	adj vs FD max rel err	$< 10^{-6}$	$< 10^{-6}$
E0a	tape vs FD max rel err	$< 10^{-6}$	$< 10^{-6}$
E0a	adj vs tape max rel err	$< 10^{-9}$	$< 10^{-9}$
E0a	FD / adjoint cost ratio	$30\text{--}80\times$	$\sim 35\times$
E0b	dot-product test rel err	$< 10^{-12}$	$\sim 10^{-17}$
E0b	factorization reuse ratio	$> 1.2\times$	$> 1.2\times$
E0b	misfit drop (40 GD steps)	$> 100\times$	$> 100\times$
E0c	heat-chain adj vs FD	$< 10^{-5}$	$< 10^{-5}$
E0c	store vs recompute dJ agreement	$< 10^{-10}$	~ 0
E0c	backward/forward wall ratio	$< 2.5\times$	$< 2.5\times$
E0c	logsumexp/max gradient-jump ratio	< 0.5	≈ 0.31
E0c	signal / FD-floor ratio	$> 10^3$	$\approx 10^4$
E0c	flat-J probe contrast	$> 10\times$	$\approx 100\times$
E0c	AC three-way (adj/cstep/FD)	$< 10^{-6}$	$< 10^{-6}$

References

- [1] P. E. Farrell, D. A. Ham, S. W. Funke, and M. E. Rognes, *Automated Derivation of the Adjoint of High-Level Mathematical Programs*, SIAM Journal on Scientific Computing **35** (2013) C369–C393. <https://doi.org/10.1137/120873558>
- [2] S. K. Mitusch, S. W. Funke, and J. S. Dokken, *dolfin-adjoint 2018.1: automated adjoints for FEniCS and Firedrake*, Journal of Open Source Software **4** (2019) 1292. <https://doi.org/10.21105/joss.01292>
- [3] A. Griewank and A. Walther, *Algorithm 799: Revolve: An Implementation of Checkpointing for the Reverse or Adjoint Mode of Computational*

Differentiation, ACM Transactions on Mathematical Software **26** (2000) 19–45. <https://doi.org/10.1145/347837.347846>

- [4] DiffSim group, *Beyond Flory-Huggins: differentiable $S(q)$ instrument-space observable, gauge identifiability, and composition-coverage conditioning* (internal dev note, 2026-07-14). [docs/dev/2026-07-14-beyond-fh-learned-thermo.md](#).

A The Adjoint-Readiness Checklist

Apply this checklist to any new PDE before writing gradient code. The normative, citable version lives on the reference page [docs/theory/adjoint_readiness_checklist.md](#) (T4). This appendix is the derived, annotated form; the reference page owns the normative list.

1. What to tape. Tape only the residual kernels that depend on the parameters you differentiate with respect to. Record $R(u, m)$ in m (and in u if a later step or J needs $\partial J / \partial u$). Everything algebraic-and-parameter-dependent is taped; the linear solver is not (item 4).

2. What to freeze.

- Mesh, octree, element classification, preflight checks — piecewise-constant in parameters (§7, item 3).
- Basis and quadrature tables: frozen inputs (§7, item 4).
- RNG seeds: reparametrize.
- Any GP-field precomputed outside the tape: the M2 “frozen at GPs” convention (§8, item iii).

3. What to store across steps. A transient adjoint sweeps backward and contracts with the primal state u^n at each step (§10). Store the whole trajectory (full-store, $O(N \times n)$ memory) or store checkpoints and recompute segments (revolve, $O(\log N)$ memory at $O(N \log N)$ recompute cost). Print the storage inventory before committing to a strategy.

4. Transposed-solve infrastructure. The gradient path through any solver is the transposed solve at the converged state (§§5, 7). One LU factorization serves both A and $A^\top$ solves (§8). Build and test the adjoint solve before writing the optimisation loop.

5. The verification ladder.

- (a) Dot-product test $\langle Av, w \rangle = \langle v, A^\top w \rangle$ per operator (§9).
- (b) Three-way gradient check: hand adjoint vs tape (or complex-step) vs central FD, for a small problem. Agreement $< 10^{-6}$ relative error is the house standard.
- (c) Transient-chain check: the full backward sweep vs FD end-to-end (§10).

Climb the ladder before trusting a gradient in an optimiser.

6. Nondifferentiability hazards. Replace `abs`, `min`, `max`, `threshold`, and `mask` operations with relaxed forms (§12): `softabs` ($\sqrt{x^2 + \varepsilon^2}$), `logsumexp` (21), sigmoid gates. Do this *before* optimising.

7. The $2.5\times$ cost contract. A correct reverse-mode adjoint adds $O(1)$ solves per forward step. Measure the backward/forward wall-time ratio (§11) before scaling up. If backward $\gg$ forward, something is being unrolled that should not be.

J-design addendum. Once the adjoint is verified:

- *Signal scale first* (H4 protocol, §13): confirm $|\partial \hat{J} / \partial m|$ clears the FD noise floor by $\geq 10^3$ before any optimisation.
- *Probe placement matters* (§14): an observable blind to the active region gives a vanishing gradient; design the measurement to see the physics.
- *Identifiability first* (§15): compute the observation-Jacobian condition number before fitting. Remove gauge freedoms from the parameterisation by construction. Add Tikhonov regularisation as a scientific prior, not as a solver fix.
- *Diverse observables* (§16): combine probe misfits, structure-factor misfits, and integral observables to illuminate different directions of parameter space.